Simple optimization with EnergyPLAN (solar, wind, fuel share) With Domain Knowledge

February 26, 2014

1 Introduction

This report is about the chromosome representation and design of mutation based on domain knowledge. The report also describe the disadvantage of chromosome representation in binary.

2 Chromosome representation

Chromosome representation is one of the important steps in evolutionary algorithm. A good representation helps the algorithm to search the space more efficiently. For our design variables, we need to deal with some step size. Consider a design variable, such as Off-shore wind capacity in MW. For the least discrete case, the design variable may differ to one another by 1 MW (e.g., for one case the off-shore wind capacity is 5000 MW, for another design the capacity is 5001 MW). But the decision maker may not be intersected at the level of accuracy of 1 MW. So, the representation should be generalized to deal with any given step size. The state of the art can not able to deal with the any given step size. So, if the step size is 50 MW, then the design variable varies with 50 MW step size.

2.1 Problem with Integer representation

Most of the time, Integer variables are represented in binary format. But there are some problems to deal with binary representation. The first problem is the maximum limit of variable depends on number of bits used to represent the variable. As an example, if we consider 6 bits to represent the variable, we will have the limit from 0 to 2^6. On the other hand, for generalized case, we need to deal with dynamic limit. Because, for a particular region, a source of renewable energy can not go beyond a maximum capacity. However, there can be a possible binary representation that can deal with any given bound using the following formula 1. But, the representation may suffer serious problem of the generating same phenotype from two different genotypes. As an example, if we have range

[10, 30], we need 5 bits to represent the variable. A genotype $(01010)_2 = 10$ produce a phenotype 20. Another genotype $(11111)_2 = 21$ produces the same phenotype 20.

$$phenotype = lb + (genotype\%(ub - lb + 1)) \tag{1}$$

Where lb means lower bound and ub means upper bound.

So, we decided to use real representation with some rounding and some techniques to deal with a given step size.

2.2 Representation

In this optimization problem, we need to deal with two kinds of design variables. First four (later on more) variables are the capacity of four different sources in MW (Off-shore wind, on-shore wind, PV and conventional power plant). And we have another three design variables those represent the share of three fuels (coal, oil and N.Gas) mixing for conventional power plant. Each fuel share value represents relative share with respect to the use of other fuels. As an example, if 0.5 and 0.5 is given for coal and oil, EnergyPLAN[1] try to use same amount of coal and oil. But if 0.25 and 0.5 is given for coal and oil, EnergyPLAN tries to use double amount of oil than coal.

For the decision variables of the capacity of different sources, we use real representation where the maximum capacity is given. On the other hand for fuel shares, we use real representation within a range of $0, 1$. To deal with the step size we propose different repair functions for those two kinds of design variables.

2.3 Repair function

The concept of repair function [2] is vastly used in the field of evolutionary algorithm. The main concept of repair function is that repair the genes[2] those are invalid in terms of search space exploration. For example, the capacity of 1510 MW for a source is invalid in a step size of 50 MW. So, our proposed repair function takes a real number and repair the gene in such a way so that the gene maps with nearest valid gene value of the invalid value. For the above example, the repair function maps the gene value to 1500 MW. The following equations describe the function.

$$value = \lfloor |geneValue| \rfloor + 0.5$$

$$rgv = \begin{cases} value + (sts - (value \bmod sts)) & \text{if } (value \bmod sts) >= sts, \\ value - (value \bmod sts) & \text{otherwise.} \end{cases}$$

[1] It is assumed that EnergyPLAN is already introduced.

[2] Each individual may contains several genes. In our case, we have nine genes within a individual

Table 1: Fuel Share Example

fuel	coal	Oil	N. gas
gene value	0.10	0.051	0.82
Fuel share before repair	0.103	0.053	0.844
gene value after repair	0.097	0.097	0.78
Fuel share after repair	0.1	0.1	0.8

Where rgv is the repaired gene value and sts is the step size.

We want to continue with the same idea for mixing of fuel shares. Because most of the time, decision maker are not interested in exact percentage (e.g.: 33% coal, 23% oil and 44% natural gas). So, we want to employ the concept of step size, where decision maker can define the step size. But there are some problem to use the same idea in fuel share. The way EnergyPLAN treat the fuel shares require some new modification of the earlier concept. It is already mentioned that fuel share in EnergyPLAN is treated as relative fuel share with respect to other fuel share. Each gene of each fuel has a limit between 0 to 1. So, the repair function we use here adjust the the gene values in a way so that relative shares of each fuel will be a multiple of step size. An Example is given in the table 2.3 where step size is 0.1 (i.g.: 10%). The 1st row of the line is gene values for fuel share for a particular individual. The 2nd row is the shares calculated using (2). 3 rd and 4th rows present the same things after repairing the gene values.

Share for a fuel is defined by the following formula.

$$fuelShare_i = \frac{geneValue_i}{\sum_{i=1}^{n} geneValue_i} \text{where } i \in Coal, Oil, N.gas \tag{2}$$

Now, we present an algorithm to repair the gene value according to the step size given by a decision maker 2.

The repair of fuel genes are more complicated than the repair of sources' capacity genes. The values represent in the fuel share genes do not represent the absolute shares of the fuels. It represent the relative share of the fuels. So, a two step process can solve the problem. In the 1_{st} step, actual fuel share is calculated (step # 2 and 3 of algorithm 1) and adjust the absolute shares according to the given step size (Step # 4). The adjustment is done by using the same concept of adjusting the values of the capacity of electricity generating sources 2.3. Then, in the 2nd step, the new relative shares are assign according to the fixed absolute share (Step # 7). Step #5 and #6 is necessary to make some minor change to the gene, if sum of all gene are not equal. Sometime this may be happen because of rounding effect. For the same rounding effect, it may be possible to have a gene value greater than 1. In that case step #8 and #9 are require to scale down the gene values.

A step-wise example is given in table.2. And positions of use of the repair function within the main evolutionary loop is shown in fig. 2.3.

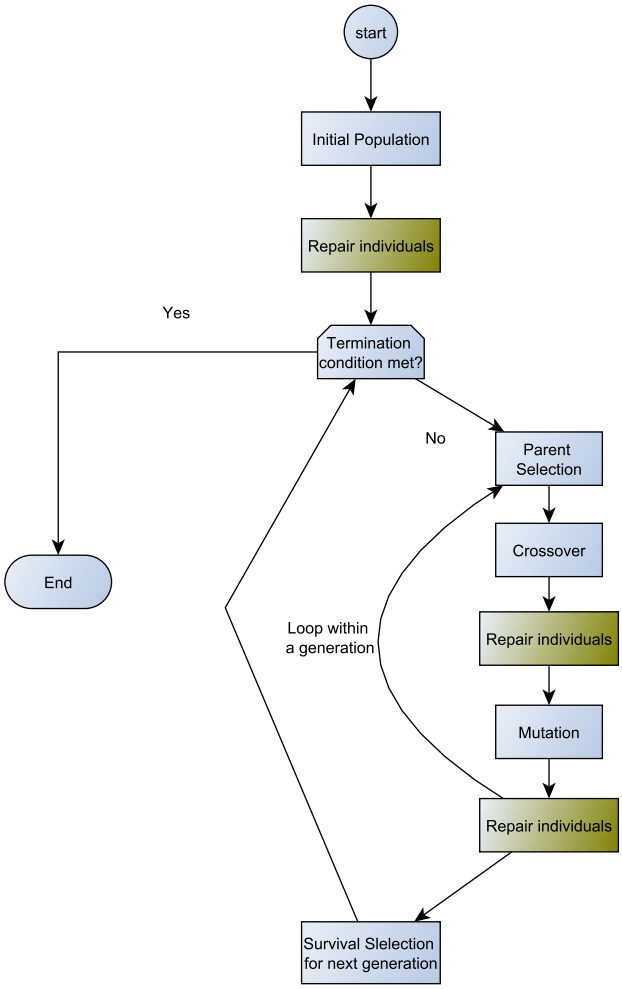


Figure 1: Position of applying repair function within the main loop of evolutionary algorithm

Algorithm 1 Repair Function for fuel genes

```
function REPAIR FUEL GENE(StepSize, fuelGenes)
    originalSum = sum of all values of fuelGenes
    normalize each gene value according to originalSum
    fixedGeneValues=adjust each gene value by stepSize
    if sum of all the scaled gene value is not equal to '1' then
        adjust a randomly gene value to make the sum equal to '1'
    newGeneValues = OrinialSum * fixedGeneValues
    if any new gene value is greater than 1 then
        Scale down each gene value by maximum gene value
    return newfuelGenes
```

Table 2: An step (mentioned in the algorithm 1) wise example of repair function for fuel share

Description	coal	Oil	N. gas
Original Value	0.104173	0.051276	0.98
Step # 3	0.091746	0.045159	0.863095
Step # 4	0.1	0	0.9
Step # 7	0.113545	0	1.021904
Step #8, 9	0.111111	0	1

3 Problem Specific Mutation

Mutation is one of the key operators in evolutionary algorithm. Here we present a problem specific mutation specially designed to deal with the optimization problem of renewable energy and conventional energy. The mutation operator present here is a combination of three different mutations. The operator can chose one of three different mutations. Depending on a random value, the mutation sometimes favours renewable sources to increase their capacity and use cleaner fuel, sometimes the mutation do completely the opposite (i.e.: favours conventional sources and low cost fuel). To keep the diversity of the population, sometimes the mutation follow the conventional mutation strategy (apply Polynomial mutation or non-uniform mutation). Fig. 3 illustrates the overall mutation process (in the fig., P(RFM) is the probability of applying the Renewable energy favour mutation and P(CEFM) is the probability of conventional energy favour mutation). Without the use of conventional mutation approach we may lead the search toward extreme solutions. In that case, we will lose solutions those may very interesting to the decision makers because the solutions may be very good as trade-off solutions.

To design the mutation that favour renewable energy and conventional energy, the following assumptions are considered.

- It is better to increase the capacity of renewable sources to reduce CO_2 emission

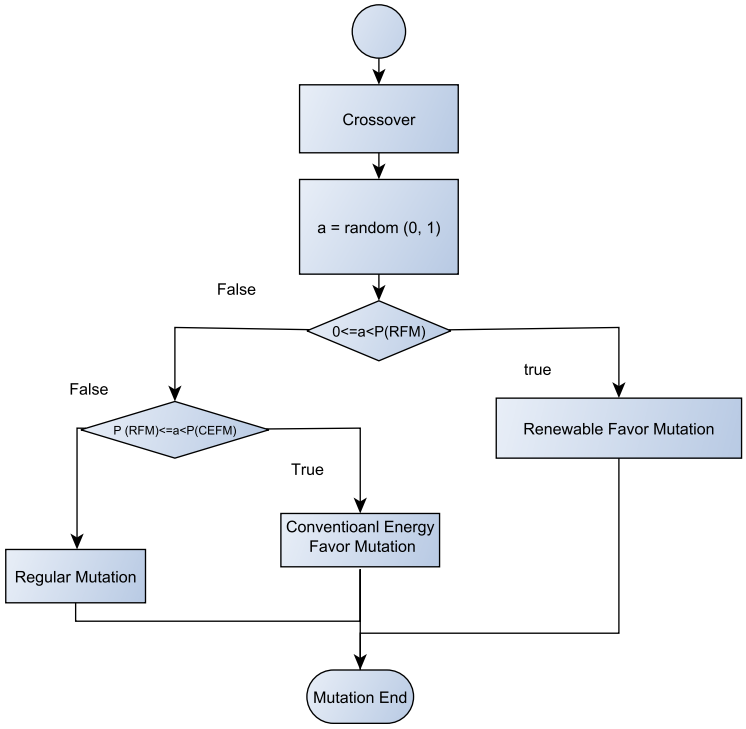


Figure 2: Mutation Flowchart

- To reduce the annual cost, the use of coal should be increase
- The use of coal should be decrease to reduce CO_2 emission
- Natural gas is costly but clean fuel.

In the next chapter, we present the mutation that favour renewable energy. Another mutation that favour conventional energy is the same as the previous one expect the chose of decision variable. But the mutation process that apply on the variables is the same.

3.1 Renewable energy favour mutation

We want to design a mutation operator that can favour renewable energy sources. Traditional mutation operator makes a change to the individual so that the individual may fall both in the upper side or lower side of the individual (e.g.: red line of the left figure of fig.3.1). But to favour both renewable, we want the mutation operator should make a change to some chosen decision variable so that the new individual should only be on upper or lower part of the old

individual (e.g.: blue line of the left figure of fig.3.1) As an example, one gene value of an individual is **1250**, and the upper and lower limit of the gene is 0 and 2000. And if the gene represents the capacity of a renewable source and we want to favour Renewable sources, the mutation operator only generate a new individual within the range of 1250 to 2000. If we want to favour conventional sources, the mutation should produce a new individual within the range 0 to 1250.

We use the concept of *normal distribution*[8] to design the mutation. The reason behind selecting normal distribution is that it give better control over the distribution through two parameter, *mean and standard deviation.* The idea used here is that we will have a normal distribution centered around the gene value of an individual(mean, μ is set equal to gene value and σ is set to $d/3$, where $d = |geneValue - geneUpperValue$ or $geneLowerValue|$). The selection of σ value ensures that most of the time the new gene value will lie near to the gene value (i.e: mean value). Then the probability density function (pdf) is truncated within a range from gene value to gene upper/lower value [10]. The red distribution in the left side of fig. 3.1 shows the pdf distributed from $[-\infty$ to $\infty]$ with $\mu = 3$ and $\sigma = 1.0$, where gene value is 3 and upper bound of the gene is 6. In the picture, gV represents the gene value and gUB represents the upper bound of the gene. On the other hand, the blue distribution shows the truncated pdf within the range of 3 to 6. And the corresponding cumulative distribution function is also shown in the right side of the figure. A random number is drawn from a uniform distribution within a range of 0 to 1. A new gene value is got using *inverse cdf of quantile function*[9]. The detail of the process is given in the following algorithm 2.

Mutation	off-Shore Wind	On-shore Wind	PV	PP	coal	Oil	N. Gas
favour RE	✓	✓	✓	–	✗	–	✓
favour CV	✗	✗	✗	–	✓	–	✗

Table 3: Different sources that increase or decrease in two different mutation. ✓indicate the increase and ✗indicate decrease, – indicate doing nothing

The above table 3.1 shows the variables that are increased or decreased according to the mutation status.

4 Quality Indicators for Multi-objective Optimization Problem

The way of comparing two algorithms in multi-objective optimization are completely different than single objective optimization. For a single objective optimization case, a algorithm is preferable than another, can be simple identified by measuring the value of objective function that need to be maximized or minimized. On the other hand, for a multi-objective optimization case, the comparison is not so simple. Unlikely the single objective case, multi-objective

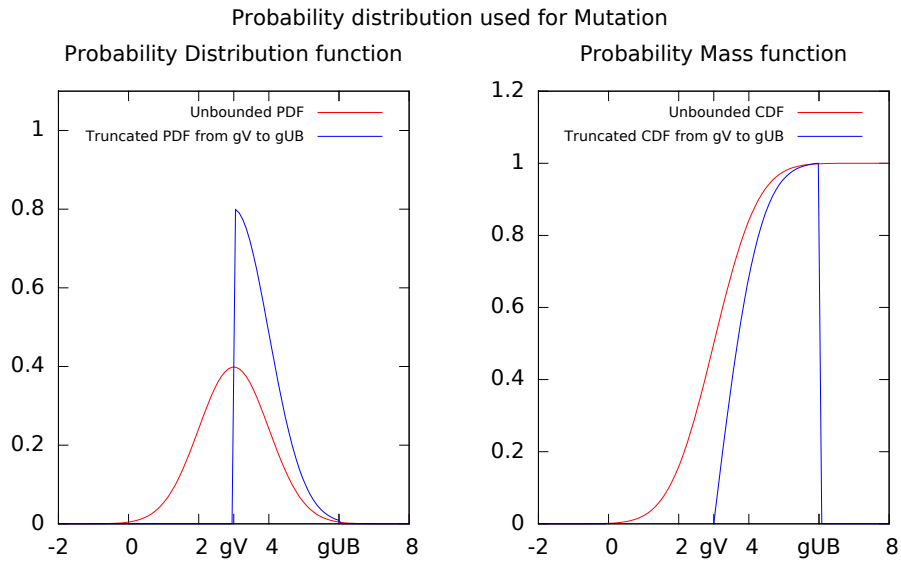


Figure 3: An example of Probability density function and probability mass function used in mutation

Algorithm 2 Mutation favours Renewable sources

function MUTATIONFAVOURSRES($individual, ListOfFoveredGene$)
  for each gene of individual **do**
    if gene $\in ListOfFoveredGene$ **then**
      Define a normal distribution $\mu = gV$, $\sigma = d_u/3$
      Truncate the distribution where distribution lower bound $= gV$ and distribution upper bound $= gUB$
      Calculate the cdf of the probability distribution function
      Take a random number uniformly distributed over 0 to 1
      $gV =$ inverse cdf (quantile function) for the random number
    else
      Define a normal distribution $\mu = gV$, $\sigma = d_l/3$
      Truncate the distribution where distribution lower bound $= gLB$ and distribution upper bound $= gV$
      Calculate the cdf of the probability distribution function
      Take a random number uniformly distributed over 0 to 1
      $gV =$ inverse cdf (quantile function) for the random number
  return *individual*

Where gV is the gene value, gLB is the lower bound of the gene , gUB is the upper bound of the gene, $d_u = gUB - gV$ and $d_l = gV - gLB$.

optimization algorithm generates a set of non-dominated solutions called Pareto optimal set. Also the assessment criteria of an algorithm is depends the user demands. As an example, a user may be in interested on an algorithm that approximate global Pareto-front consistently, rather than an algorithm that converge to global Pareto-front but occasionally. The quality of a Pareto-Optimal set generated by a method/algorithm mainly depends on three factors [6].

1. Minimize the distance between true Pareto-front and obtained non-dominating set.
2. Maximizing the Spread of the solution in the set.
3. Maximising Number of elements in the set

Many literatures are published to capture the all three criteria into a single value. In fact, no indicator really able to capture all three issues discussed above within a single value [6]. Actually interesting enough to mention that the issues discuss above are contradictory to each other and the design of a assessment indicator itself is a multi-objective optimization problem. Therefore, it is not worthy to make a decision for an algorithm based on single assessment indicator.

Here, some of the main quality indicators found in literature are described below.

4.1 Generational Distance (GD)

The measurement of generational distance (GD) is done by calculating the distance between the elements of achieved Pareto front and known true Pareto front (TPF) [7].

$$GD = \frac{\sqrt{\sum_{i=1}^{n} d_i^2}}{n} \tag{3}$$

where n is the number of element found in a achieved non-dominated set and d_i is the Euclidean distance between the points in found non-dominated set and nearest points of TPF (measured in objective space). The closer the value of GD to 0 means the obtained Pareto-front is closer to known true Pareto-front. In the case of $GD = 0$, the achieved Pareto front exactly match with TPF. In the picture 4.1, the red lines represent the distance between the non-dominated points and true Pareto-front points. The main problem of this quality indicator is that it value of the indicator may be misleading, if there are two different non-dominated sets need to be compared, and one set has very few points close to the TPF points and another set has many non-dominated points including the points of earlier set. The problem also exists if the two sets have same number of points. Consider, a set has 50 points those are very compact and close to each other and also close to TPF, on the other hand, another set has same number of points but they are well distributed but not so close of the TPF. In the field of multi-objective optimization distribution of non-dominated points is an important issue to be considered.

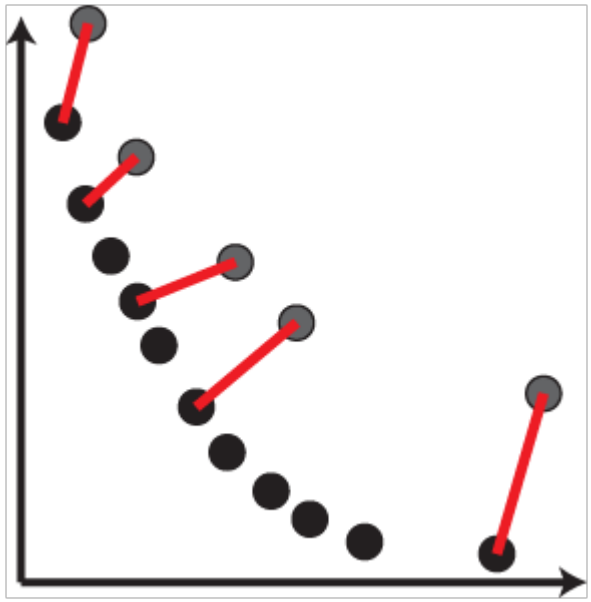


Figure 4: Generational Distance

4.2 Inverted Generational Distance (IGD)

To deal with the problem of GD, a very similar quality indicator is proposed, called inverted generational distance(IGD) [7]. The concept of the indicator is opposite to GD. Here, the distance from the points of TPF to the point of obtained non-dominated set is measured and averaged.

$$GD = \frac{\sqrt{\sum_{i=1}^{k} d_i^2}}{k} \tag{4}$$

where, k is the number of points in true Pareto-front and d_i is the distance between a point of TPF and the nearest point of obtained non-dominated Pareto front. A picture willl be nice. The better the quality of the solution set, the less the value of IGD will found. When $IGD = 0$, that means the solution set not only lies on the TPF, but also it covers each points of know true Pareto-front. By comparing the points of TPF with non-dominated set, IGD gets free from the problems founded in GD.

4.3 Spread

It is highly desirable that obtained Pareto-front should be well distributed and well spread to provide all possible options to the decision maker. Deb at al. [3] introduced an indicator, called *spread*, to understand how well distributed

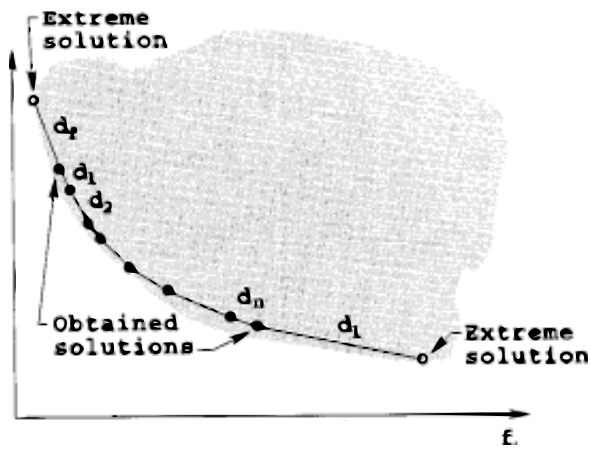


Figure 5: Calculation of Spread Indicator (taken from [3]

the front is. The indicator measures the span of the achieved non-dominated solution set. The indicator is measured by taking Euclidean distance between consecutive solutions and make a average of those distances. Then another two Euclidean distances are measured from the extreme two points of TPF and boundary points of the obtained set. All the distances are measured in objective space. Finally using the following formula 5, the value of the Spread is measured the non-uniformity in the distribution.

$$\Delta = \frac{d_f + d_l + \sum_{i=1}^{N-1} |d_i - \bar{d}|}{d_f + d_l + (N-1)\bar{d}} \tag{5}$$

Where, d_f and d_l are the Euclidean distances between the extreme solutions of TPF and boundary solutions of achieved set 4.3. $\bar{d}$ is the average distances over all d_i, assuming that there are N number of solutions and $N - 11$ number of consecutive distances in the obtained set. A good distribution will have d_i value close to $\bar{d}$ and d_f and d_l equal to zero, make the indicator value, Δ, as close to zero.

4.4 Generalized Spread

The indicator is introduced by Zhou A. [11]. The indicator is the extension of previous indicator of *spread* to work with more than 2 objectives.

$$\Delta(S, TPF) = \frac{\sum_{i=1}^{m} d(e_i, S) + \sum_{x \in TPF} |d(x, S) - \bar{d}|}{\sum_{i=1}^{m} d(e_i, S) + |TPF|\bar{d}} \tag{6}$$

Where S is the obtained non-dominating solution set and TPF is the true Pareto front, m is the number of objectives, $e_i \in \{e_1, \ldots, e_m\}$ are the m extreme solutions, $|TPF|$ is the number of solutions in TPF and

$$d(x, S) = \min_{y \in S, y \neq x} \parallel F(x) - F(y) \parallel^2 \tag{7}$$

$$\bar{d} = \frac{1}{|TPF|} \sum_{x \in S} d(x, S) \tag{8}$$

where $F(x)$ is the objective vector consists of all the m objective values for x.

4.5 Epsilon

This indicator is first proposed by Zitzler at al [13]. Here the concept of domination of one set to another set is considered in all objectives. Considering two sets, the concept is that how much minimum distance a bad set needs to be translated to dominate the better set. In the figure 4.5, the set coloured with black is better than the gray set for the minimization problem. The *red line* indicate the amount, that much the gray set needs to translate to be able to dominate black set. Mathematically,

$$I^1_{\epsilon_+}(S, TPF) = \inf_{\epsilon \in thbbR} \{\forall \vec{p} \in TPF \mid \exists \vec{s} \in S : \vec{s} \preceq \vec{p} + \epsilon\} \tag{9}$$

Where $\vec{s} \preceq \vec{p}$ means $\vec{s}$ dominates $\vec{p}$.

4.6 Hypervolume

This indicator/metric is also proposed by Zitzler [12] in 1999. The indicator measures the volume covered by the solutions of non-dominated set in objective space. For bi-objective optimization problem, each solution is covered by a rectangle from a reference point (for maximization problem, the reference point is generally origin, and for minimization problem reference point can be chosen from the worst solution with all population). The metric is measured by doing union of all rectangles founded from the solution of obtained non-dominated set. Mathematically,

$$HV = volume \left(\bigcup_{i=1}^{|Q|} v_i \right) \tag{10}$$

Where Q is the set of non-dominated solutions, for each solution in Q, a hypercube, v_i is created from the reference point. And the volume of the union of all hypercubes measures the value of the hypervolume. Figure 4.6 shows the union of all rectangles for a set.

The main drawback of the indicator is, the value can be misleading if the shape of the set in objective space is concave [6]. The indicator may provide poor measurement if two algorithms may differ more than one issues discussed before 4.

It is worth to mentioned that all sets need to be normalized to get those matrices.

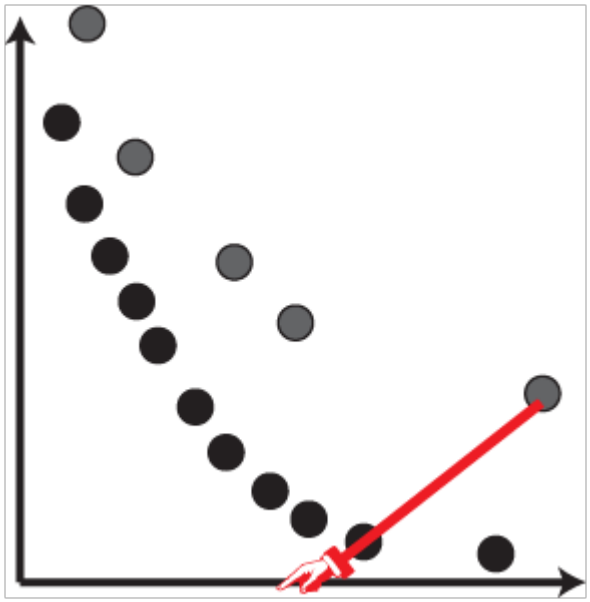


Figure 6: calculation of epsilon indicator

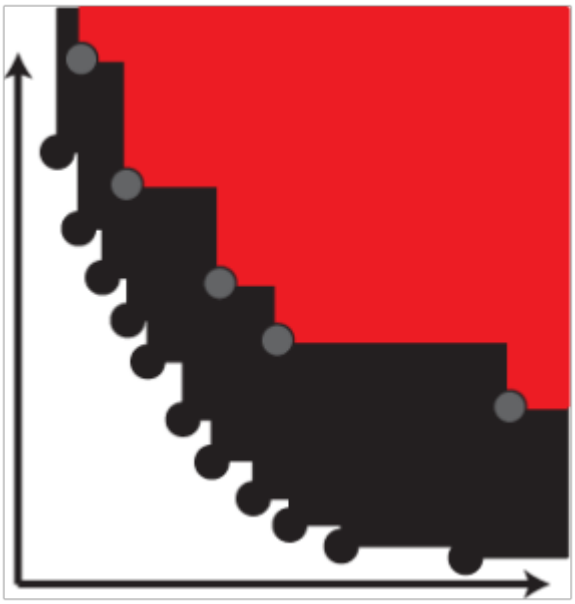


Figure 7: calculation of hypervolume indicator

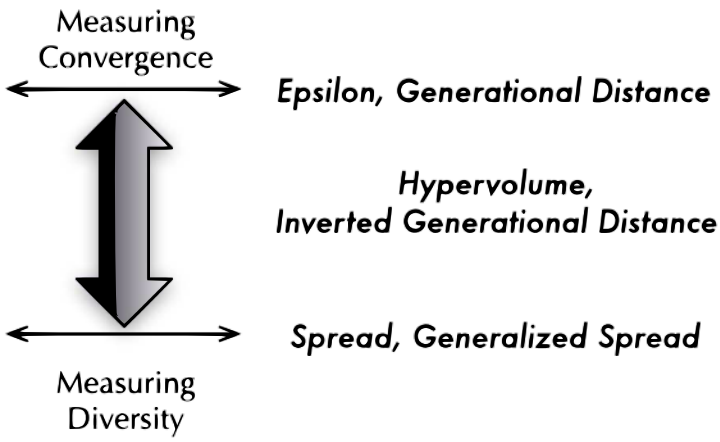


Figure 8: Indicators Category (taken from [1])

4.7 Discussion

It is clear from above discussion that no single metric or indicator is able to provide us clear indication about an algorithm/method. The decision makers generally want a number of uniformly distributed solutions over tread-off surface. So, the comparison of a combination of some indicators is a good chose to compare algorithms [6]. According to the discussion, the indicators can be categorizes into three different categories [1, 5].

- Convergence: GD & Epsilon
- Diversity: Spread & Generalized Spread
- Convergence & Diversity: Hypervolume & IGD

So, in our experiment, we will use a combination of quality indicators to evaluate our algorithms. *Hypervolume, Inverted generational distance* with *epsilon* for convergence and *spread* for diversity will be used.

5 Experimental Setup

To test the performance of our developed mutation we have done two kinds of experiments. The experiment is done on a dataset of Denmark. The first experiment is done using regular crossover and mutation operator (Simulated Binary Crossover and Polynomial mutation) with repair function. Next experiment is done using the same crossover but using our mutation. To have a fair competition between two methods, we use same initial population. So, two algorithm starts from same initial individual. The generation of initial population is controlled by predefined seeds. Each experiment is carried out for 10 different random seeds. Each experiment consists of 10 independent runs. Therefore,

we have carried out 20 independent runs, 10 with state-of-the-art operator and another 10 runs with our developed operator.

The following parameters are used for first experiment where we use state-of-the-art operators.

- Population: 100
- Evolution: 5000
- crossover Probability: 0.9
- Mutation probability: 0.2
- Distribution index: 4
- Algorithm: NSGA-II
- Parent selection: Binary Tournament.

To perform next experiment, we have just change the mutation probability 0.2 to 0.4. Our assumption is that more use of the mutation operator will give use better results.

To measure the performance of the operator we need to measure the value of quality indicators describe in 4. But expect hypervolume, to measure the value of all other quality indicators require known true Pareto-optimal front. As the Pareto-optimal set is not known, we construct the Pareto-front by combining the results from 20 different run. The approach is of combining the results of different is a well known practice in the field of evolutionary algorithm. when true Pareto-front is unknown [4].

After getting the true Pare-front, we have done the experiment again to get generational data for quality indicators. The reason behind to run the entire experiment again is that to capture the values of indicator in each generation. And to have values in each generation, we need the known Pareto-front constructed in previous experiment. Table. 5 summarize the whole experiments and purpose of the experiment.

Experiment Name	Operators	No of runs	Purpose
Exp-Regular 1	SBX & Polynomial	10	Generate true Pareto-front
Exp-DKMutation 1	SBX & DKMutation	10	
Exp-Regular 2	SBX & Polynomial	10	collect generational data for indicators
Exp-DKMutation 2	SBX & DKMutation	10	

6 Results

The results presented here divided into two part. In first part, the results are gather after completing of the run. This results presents the behavior of the algorithm (operator) at the end. In the second phase, we try to capture the

behaviors of the algorithm during the run. For this purpose, we collect the data about different indicators in each generation for different run.

6.1 Results after completion of run

The Table. 6.1 below shows the results of 10 different run for Exp-regular 1 and Exp-DKMutation 1. The shadowed cells represents better performance in the corresponding indicator. It is noted that higher value of hypervolume represent better performance and lower value of IGD means better performance. When considering hypervolume, regular operators are manage to provide good performance 2 times more then the Domain-knowledge mutation. But in respect to IGD, our mutation manage to outperform the regular operator 6 times.

	HyperVolume		IGD	
Seed	Exp. Regular 1	Exp. DKMutation 1	Exp. Regular 1	Exp. DKMutation 1
365652	0.835233	0.850046	0.003717	0.002392
455875	0.889545	0.871887	0.006704	0.001743
456545	0.889545	0.837827	0.001859	0.002354
458478	0.827764	0.840518	0.007353	0.00231
545782	0.83592	0.839544	0.002697	0.002235
547945	0.883954	0.838292	0.006959	0.002266
549235	0.841302	0.833172	0.003796	0.004549
562366	0.890897	0.840934	0.006725	0.002228
652262	0.828183	0.873807	0.007784	0.002266
981354	0.888231	0.83999	0.003411	0.002288

The next table 6.1 shows the results of Spread and generalized Spread indicators. It should be noted that each of the indicators need to be minimized to present better result. It is already mentioned earlier that *generalized spread* is a multi-dimensional version of *spread* where spread works only for two objective case. Our problem is a two objective problem and still we observed that the values of two indicators are different. There are mainly two reasons behind getting different values. The first reason is that *spread* indicator do not consider duplicate solutions, if duplicate solutions exits in obtained Pareto-front. As spread measures the distance of successive solutions, the distance between successive solutions are considered as zero. On the other hand, as *generalized spread* considers the distance of nearest solutions whose distance is greater than zero, the duplicate solutions are considered as independent solutions. The second reason is that *spread* consider each gap between all the solutions. On the contrary, *generalized spread* consider the smallest gap in any direction. Consider an one dimensional case, where three solution have values of $10, 12$, 16 and 18. If we consider spread, there are 3 distances of $2, 4, 2$. On the other hand, if we consider generalized spread, the three distance is $2, 2, 2$. We are not able to tell which on the better, but with our mutation, we have manage to outperform regular mutation 15 times within 20 times.

	Spread		Generalized Spread	
Seed	Exp. Regular 1	Exp. DKMutation 1	Exp. Regular 1	Exp. DKMutation 1
365652	0.657685	0.773247	0.573008755	0.558847165
455875	0.675538	0.785794	0.677625004	0.569152299
456545	0.675538	0.641181	0.689310642	0.385132845
458478	0.685045	0.648291	0.715928387	0.433595939
545782	0.619087	0.586206	0.544114878	0.419384069
547945	0.827126	0.544189	0.816414887	0.433969669
549235	0.644144	0.648021	0.552251426	0.636379722
562366	0.719552	0.515242	0.704807006	0.45772625
652262	0.70084	0.964505	0.697327808	0.587781616
981354	0.967737	0.618592	0.606944707	0.412998768

The following table 6.1 shows the results of *generational distance* and *epsilon* indicators. The results shows that it is a tie for epsilon, but in generational distance regular operator outperform our operator. Most probably the reason it that our operator able to produce more diverse solution but the state-of-the-art operator produce results near to true Pareto front.

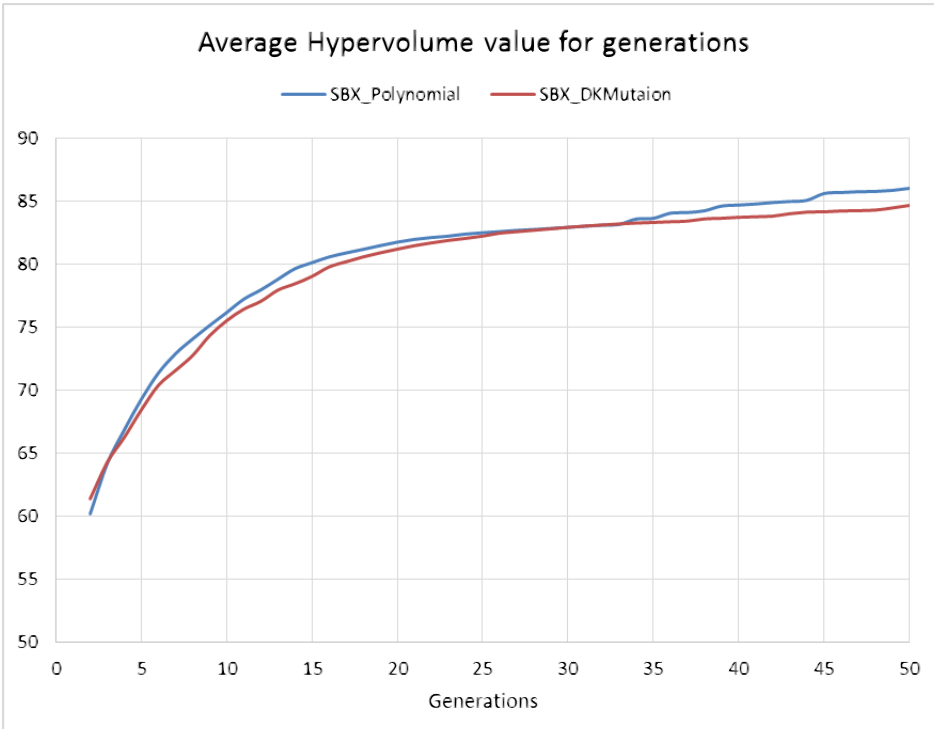
	Generational distance		Epsilon	
Seed	Exp. Regular 1	Exp. DKMutation 1	Exp. Regular 1	Exp. DKMutation 1
365652	0.005615	0.00485	1573	754
455875	0.000641	0.002132	631	5.696
456545	0.000811	0.006319	596	694
458478	0.002539	0.005513	1842	689
545782	0.00578	0.004896	1432	797
547945	0.001582	0.005812	604	1008
549235	0.005617	0.005303	1051	1688
562366	0.000489	0.006442	629	652
652262	0.002267	0.003865	1841	689
981354	0.002626	0.005827	683	702

6.2 Results during run

In this section we are going to generational results for two most important quality indicators, *hypervolume and inverse generational distance(IGD)*. To understand the overall behavior, we make an average and normalized the data for each generations.

$$HV_{gen_i} = \frac{\sum_{j=1}^{k} HV_j^i}{k} * 100 \tag{11}$$

Where i is the generation number, k is the total number of runs with different seeds. In our case, k is equal to 10. Using the equation we measure the average



value of hyperpervolume in a scale of 100 for regular operator and developed operator.

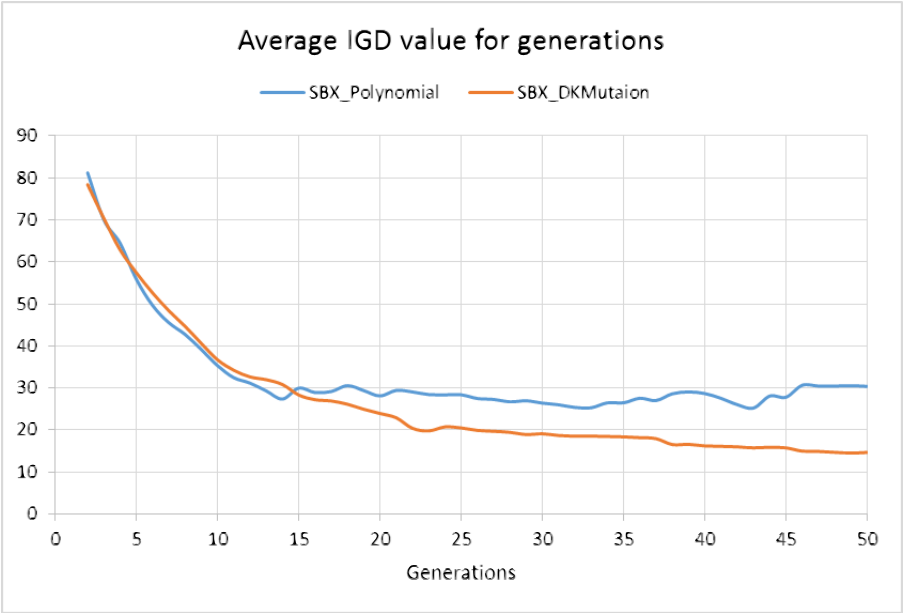
For IGD, it is expected that the value will be close to zero.

$$IGD_{gen_i} = \frac{\sum_{j=1}^{k} IGD_j^i}{k * IGD_{max}} * 100 \tag{12}$$

Where i is the generation number, k is the total number of runs with different seeds. IGD_{max} is the maximum IGD found in all different runs.

From the chart of hypervolume, it is clear that our developed mutation closely follow the generational hypervolume values of regular mutation from the beginning until end. There is not much difference from curves presents hypervolume for two different operators.

On the other hand, the chart of IGD is similar until 15 generation. After 15 generations, the curve represents DKMutation continue goes downward, but the other curve that represents polynomial mutation stabilizes.



7 Future work

The concept of favouring renewable energy sources, clean energy sources and favouring the conventional energy sources, low cost fuel can be applied to crossover operator. As a future work, these concepts may be applied to the state-of-the-art operators. For example, we can use the same distribution used in polynomial mutation using instead on normal distribution. Our primary assumption is that the above concept perform better because of instead of two different distribution we can use the same distribution.

References

[1] Guillermo Molina Arribere. Tcnicas de optimizacin para redes de sensores (optimization techniques for wireless sensor networks).

[2] P.P. Bonissone, R. Subbu, N. Eklund, and T.R. Kiehl. Evolutionary algorithms + domain knowledge = real-world evolutionary computation. *IEEE Transactions on Evolutionary Computation*, 10(3):256–280, Jun 2006.

[3] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, April 2002.

[4] J.J. Durillo, A.J. Nebro, and E. Alba. The jmetal framework for multi-objective optimization: Design and architecture. *2010 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8, Jul 2010.

[5] Siwei JIang and Jie Zhang. Consistencies or contradictions of performance metrics in multiobjective optimization? *IEEE Transactions on Systems Man and Cybernetics Part C (Applications and Reviews)*, 2014.

[6] Ruhul Sarker and Carlos A. Coello Coello. Assessment methodologies for multiobjective evolutionary algorithms. In *Evolutionary Optimization*, number 48 in International Series in Operations Research & Management Science, pages 177–195. Springer US, January 2002.

[7] David A. Van Veldhuizen and Gary B. Lamont. Multiobjective evolutionary algorithm research: A history and analysis. Technical report, 1998.

[8] Wikipedia. Normal distribution — wikipedia, the free encyclopedia, 2013.

[9] Wikipedia. Quantile function — wikipedia, the free encyclopedia, 2013.

[10] Wikipedia. Truncated distribution — wikipedia, the free encyclopedia, 2013.

[11] Aimin Zhou, Yaochu Jin, Qingfu Zhang, B. Sendhoff, and E. Tsang. Combining model-based and genetics-based offspring generation for multi-objective optimization using a convergence criterion. In *IEEE Congress on Evolutionary Computation, 2006. CEC 2006*, pages 892–899, 2006.

[12] E. Zitzler and L. Thiele. Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999.

[13] E. Zitzler, L. Thiele, M. Laumanns, C.M. Fonseca, and V.G. da Fonseca. Performance assessment of multiobjective optimizers: an analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003.